

An Efficient, Reliable and Observable Collective Communication Library in Large-scale GPU Training Clusters

Mingjun Zhang¹, Xiaohe Hu^{1,3}, Menghao Zhang², Ziteng Chen^{1,4}, Yanmin Jia¹, Yan Zhang¹, Da Liu¹, Qing Chen¹, Fangzheng Jiao², Jun Chen¹, He Liu¹, Aohan Zeng^{4,5}, Shuaixing Duan⁵, Ruya Gu¹, Yang Jing¹, Bowen Han⁶, Wei Chen¹, Wenqi Xie¹, Jinlong Hou³, Yuan Cheng³, Hongzhou Zhang⁷, Bohua Xu⁶, Mingwei Xu⁴, Chunming Hu²

¹*Infrawaves* ²*Beihang University* ³*Shanghai Innovation Institute* ⁴*Tsinghua University*
⁵*Zhipu AI* ⁶*China Unicom Research Institute* ⁷*Shanghai AI Power Technology Co., Ltd*

Abstract

Large-scale LLM training requires collective communication libraries to exchange data among distributed GPUs. As a company dedicated to building and operating large-scale GPU training clusters, we encounter several practical limitations of NCCL in production, including 1) SM competition between computation and communication, 2) expensive restart costs under link failures, and 3) insufficient observability of transient collective communication anomalies. To address these challenges, we propose VCCL, an efficient, reliable, and observable collective communication library in large-scale GPU training clusters. VCCL removes SM-consuming P2P kernels by moving intra-node data movement and stream dependency enforcement to CPU threads and GPU copy engines. VCCL also introduces a primary-backup QP mechanism to tolerate frequent NIC port failures, and designs a window-based monitor to observe network anomalies at $O(\mu s)$ level. We open-source VCCL and deploy it in production training clusters for several months. Compared with NCCL, VCCL improves training throughput by up to 5.28% and reduces massive GPU resource wastage through runtime fault tolerance and fine-grained monitor. We also share experience and lessons we learned during the deployment of VCCL in large-scale clusters.

1 Introduction

Collective communication (CC) plays an essential role for data transmission among GPUs in distributed training of large language models (LLMs). CC libraries (CCLs) offer various collective primitives and utilize the intra-host and inter-host interconnections, whose communication efficiency is of great importance in large-scale LLM training systems. Given the dominant market share of NVIDIA GPUs, NVIDIA Collective Communication Library (NCCL) [10], specially tailored

and optimized for NVIDIA GPUs, has become the *de-facto* CCL in mainstream large-scale GPU training clusters.

As a company dedicated to building and operating large-scale LLM training clusters, we strive to provide the most cost-efficient LLM infrastructure for our customers. However, when dealing with CC for large-scale training clusters, we encounter three main challenges with practical usage of NCCL.

Challenge 1: SM competition between computation and communication in P2P primitives. GPUs offer high parallelism for general matrix multiplication (GEMM) and value reduction, allowing programmers to write kernels and use GPU streaming multiprocessors (SMs) to accelerate computation. In NCCL, however, we observe the point-to-point (P2P) primitives (e.g., `SendRecv` and `AlltoAll`) that do not involve reductions also consume non-negligible GPU SMs, leading to SM underutilization on computation tasks. Meanwhile, NCCL P2P suffers from redundant GPU-CPU synchronization irrelevant to the actual communicating process, thereby slowing down the P2P networking performance. Consequently, NCCL compromises customers’ investment of GPU computing power to optimize LLM training efficiency.

Challenge 2: Poor tolerance to NIC port failures. In large-scale GPU clusters, accidental link failures occur frequently, especially with network speed beyond 400Gbps, leading to stragglers and unexpected crashes of CC due to timeout error. Nevertheless, NCCL lacks a native fault tolerant mechanism and is therefore unable to migrate the transmission when failures happen. This can result in significant GPU waste to relaunch CC and LLM training frameworks.

Challenge 3: Limited observability for transient network anomalies. GPU training clusters are dominated by CC workloads, many of which usually finish within $O(ms)$ and even $O(\mu s)$. Nonetheless, when network anomalies occur, NCCL lacks fine-grained observability to capture the $O(\mu s)$ -level network performance drops at runtime. It prevents operators from promptly localizing the anomalous links, which is essential to provide proofs for “network innocence”.

While some existing studies from industrial communi-

Mingjun Zhang and Xiaohe Hu contributed equally to this paper. Menghao Zhang is the corresponding author (zhangmenghao@buaa.edu.cn).

ties [1, 18, 19, 24, 29, 49, 57] have shared their experience in large-scale LLM training, none of them presents a comprehensive and systematic perspective on an ideal CC service with optimized efficiency, reliability, observability in large-scale production environments. In this paper, we propose VCCL (Infrawaves Collective Communication Library), an efficient, reliable and observable CCL for large-scale GPU training clusters. Based on real-world cases and shared requirements from our customers, we present our practices for designing and operating a production-level CCL in detail. To the best of our knowledge, this is the first work that comprehensively discusses the requirements, designs and operations of a successful CCL in production environment at scale.

VCCL includes three key designs. Firstly, VCCL employs a SM-free P2P mechanism to minimize SM consumption. Specifically, it utilizes CPU-based alternatives to GPU kernels for intra-node data movement, GPU-CPU synchronization, and CUDA stream dependency management. SM-free P2P can not only spare more SM resources for GEMMs and improve the P2P performance, but also allow us to optimize the overlaps of pipeline parallelism during LLM training process. Secondly, VCCL introduces a primary-backup queue pair (QP) mechanism between each GPU pair. When a link failure happens, VCCL switches to another healthy link via backup QP, enabling runtime state migration between QPs and breakpoint retransmission to tolerate NIC port downs. Thirdly, VCCL exploits the narrow-waist abstraction of Remote Direct Memory Access (RDMA) to monitor throughput in $O(\mu s)$ granularity. It records the generation timestamps of Work Requests (WRs) and Work Completions (WCs), and utilizes a sliding window to smooth out fluctuations measured by the naive per-message scheme.

We implement and open-source VCCL at Github [8]. VCCL has been deployed in multiple large-scale training clusters that we operate, with the largest deployment scaling to 24K GPUs. Experimental results show that, compared to NCCL, VCCL delivers higher bandwidth while reducing inter-node small-message latency by 18.9% on average. Besides, real-world results show that the SM-free design of VCCL improves end-to-end training throughput by up to 5.28% against NCCL. When RDMA NIC (RNIC) port failures occur, VCCL’s primary-backup QP mechanism can maintain the majority of CC throughput and almost the same training performance as in normal conditions, with production-level data indicating that it can reduce GPU time wastage by nearly 90%. For transient network anomalies, VCCL’s window-based monitor accurately pinpoint network stragglers without misclassifying other anomalies in $O(\mu s)$ granularity with low system overhead. Furthermore, we also share several experiences and lessons when integrating VCCL with large-scale GPU clusters, hoping this work can give communities more insights on the production-level CCL in LLM training.

The main contributions of this work include:

- We identify that current CCLs fall short of realizing efficient, reliable collective communication with sufficient observability in large-scale GPU clusters (§2).
- We design VCCL, comprising a SM-free mechanism for P2P primitives, a primary-backup QP mechanism to tolerate NIC port downs, and a fine-grained monitor for inter-node communication (§3).
- We implement and deploy VCCL, with real-world results validating its efficiency, reliability and observability (§4). We also share experience and lessons we learned from deploying VCCL in real-world production GPU clusters (§5).

This work does not raise any ethical issues.

2 Background and Motivation

2.1 Preliminary

GPU Architecture and Programming Model. Modern GPUs serve as the fundamental accelerator for LLM, primarily comprising multiple SMs and High Bandwidth Memory (HBM). Each SM consists of numerous CUDA cores, registers, and shared memory to execute parallel threads. Developers leverage the CUDA toolkit [45] to submit computational tasks as kernels, which are scheduled onto SMs in the form of thread blocks. Beyond GEMM, SMs are also responsible for managing data movement and synchronization logic in communication libraries, making SM efficiency critical for both computation and coordination.

Collective communication library and RDMA. To enable high-performance data exchange between GPUs, NCCL [10] is widely adopted. NCCL detects the hardware topology (e.g., NVLink, PCIe) and establishes optimized logical channels, such as rings or trees. For inter-node communication, NCCL typically relies on RDMA [9] to bypass the CPU kernel and minimize latency. RDMA utilizes QP to allow direct memory access between remote GPUs. By posting WRs and polling Completion Queues (CQ), RDMA provides the high-bandwidth, low-latency transport layer necessary for large-scale clusters.

Distributed LLM Training. Current LLM training frameworks [48, 52, 56] support multiple parallelism strategies to enable efficient training among distributed GPUs, including: 1) Data Parallelism (DP) uses AllReduce to synchronize gradients; 2) Tensor Parallelism (TP) and Pipeline Parallelism (PP) partition model weights and layers, relying on AllGather or SendRecv for activations; and 3) Mixture-of-Experts (MoE) utilizes AlltoAll for token routing. The performance of these distributed strategies is fundamentally constrained by the underlying efficiency of SM-based coordination and RDMA-based network transmission. With increasing

scale and complexity of LLM training, CCL has become more crucial in both academia and industry [7, 10, 37, 40, 53, 54, 61].

2.2 Challenges in Large-Scale Training

Although NCCL has become the *de-facto* CCL in production-level GPU training clusters, we still encounter several challenges in real-world scenarios, falling short of several essential properties of efficiency, reliability and observability.

SM competition between computation and communication. GPUs utilize SM to execute parallel GEMMs in deep learning, but we observe that CC primitives that do not involve reduction, such as `SendRecv` used in PP, also consume non-negligible SM resources in NCCL. This phenomenon significantly hinders the effective overlapping of computation and communication. Figure 1 illustrates the basic procedure of inter-node and intra-node `send/recv` primitives of NCCL. For inter-node P2P operation, NCCL consists of three steps: (1) Sender buffer copy: sender rank launches kernels to copy data from application buffer to sender buffer, and notify the NCCL proxy thread upon chunk availability. (2) Network transmission: NCCL proxy thread posts WRs to trigger NIC doorbell, then sender RNIC transmits packets to the remote RNIC by GPUDirect RDMA (GDR) [55]. (3) Receiver buffer copy: receiver rank launches kernels to continuously move data from the receiver buffer to the application buffer as chunks arrive. On the other hand, for intra-node P2P operation, the procedure follows a similar yet slightly different two-step pattern, and the data movement is driven entirely by kernel launches: (1) Intra-node transmission: sender rank issues memory copy operation from sender buffer to receiver chunk buffer via NVLink/PCIe. (2) Receiver buffer copy.

We conduct a NCCL-Tests [42] experiment to evaluate the GPU SM utilization of `SendRecv` and `AlltoAll` workloads on 2 servers, each of which is equipped with 8 NVIDIA Hopper GPUs and 8 NVIDIA ConnectX-7 RNICs. The inter-host topology follows a 1:1 oversubscribed two-tier CLOS topology with 400Gbps links. And we use Nsight [47] and NVIDIA Data Center GPU Manager (DCGM) [46] to monitor the GPU SM utilization. Experimental data in Appendix A shows that though `SendRecv` and `AlltoAll` are reduction-free operations, NCCL still consumes non-negligible SM resources. What’s worse, SM occupation by kernels will not be freed until the entire P2P process finishes. For better training efficiency, our customers expect more GPU resources to be reserved for GEMM and to minimize SM wastes in communication tasks.

Expensive restart costs in link failures. As inter-host communication is embracing much higher bandwidth than before, we encounter more frequent downs of RNIC ports within and beyond 400Gbps network bandwidth. Figure 2 collects statistics on the failure types over 10 months in 2025 within a GPU cluster. It is obvious that the link failures, caused by optical modules and RNIC hardware, contribute the most failures

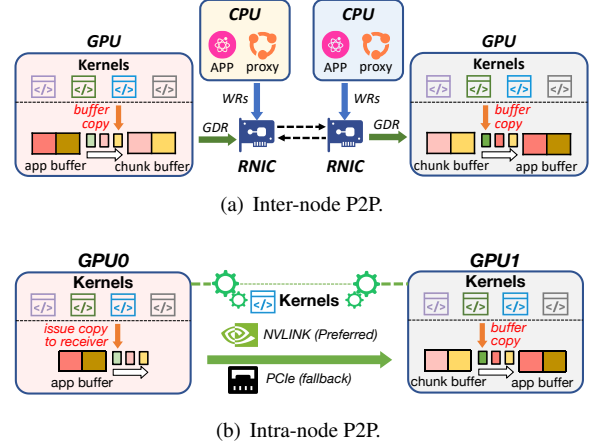


Figure 1: NCCL P2P w/ kernels.

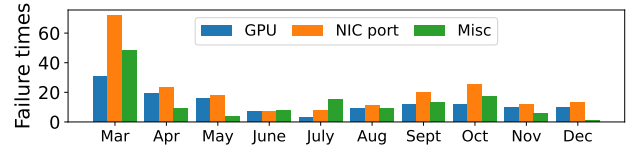


Figure 2: Failure statistics in 2025.

than GPUs and miscellaneous types.

In LLM training, link failures can lead to stragglers and suspension of the CC operation, and even crash of the entire training process by timeout exceptions. Despite the severe impact, mitigating link failures is typically straightforward, involving simple actions such as reseating optical modules or resetting RNIC driver. Unlike GPU failures, link failures do not require node isolation, theoretically allowing the original communication groups to remain intact. Nevertheless, NCCL lacks a built-in mechanism to tolerate the potential downs of RNIC ports, which introduces much time & financial costs and GPU wastes to restart the training. Therefore, our customers are seeking a lightweight fault tolerance mechanism to resist frequent RNIC port downs.

Lack of transient network observability. In LLM training, problems can occur randomly at every point. We observe that existing issues in production clusters can be categorized into two types: (1) GPU/Network failures: severe GPU/Link failures directly leading to training hang or crash. (2) GPU/Network stragglers: GPU/Network performance degradation that slows down the training process without completely stopping it. While existing diagnostics [26, 46] and previous works from industry [17, 18] effectively identify the root causes of persistent faults, they often fail to characterize transient performance degradation. Consequently, without fine-grained monitoring, network teams cannot rule out network issues as the cause of these fluctuations.

However, current network monitor systems in production clusters are based on hardware counters in $O(s)$ granularity, but most CC operations usually finish within $O(ms)$ and even $O(\mu s)$. Although NCCL Profiler [12] and Inspector [13] can provide observability for each CC, they lack visibility into

the online network performance since several CCs may flow through one NIC simultaneously, so that the tracing of single CC is insufficient to reflect network status. Therefore, our customers are demanding an observable CCL to monitor transient network anomalies.

2.3 Existing Works Fall Short

Some works have been proposed to optimize CC in different aspects. For efficiency, Centauri [5], CoCoNet [27] and Wang et al. [60] fuse kernels for fewer launches, and HFRReduce [1] offloads the reduction operation of `allreduce` from GPUs to CPUs, but none of them consider GPU SM competition between computation and communication. Recently NCCLX [57] introduces a similar SM-free design like VCCL, nevertheless, it still launches a kernel that occupies a small number of SMs to guarantee the sequential execution of stream computation and communication, which hinders training efficiency (§4.1). Additionally, NCCLX necessitates intrusive modifications to the PyTorch [48] framework, further compromising its deployability and overall usability.

For reliability, industrial practices [1, 18, 19, 29, 49] tolerate link failures by longer NCCL timeout, traffic engineering, dual-plane switches and faster checkpoints, but they either lack timely analysis & tolerance on failures, or require hardware supports that hurt deployability. NCCL itself introduces a dynamic communicator adjustment feature [15] in version 2.29. However, simple link failures do not necessarily require isolating nodes, allowing the original communication group to be maintained.

For observability, early works by Microsoft [2, 22] and ByteDance [34, 35] visualize performance anomaly with end-host probing and logging. However, these works require frequent tracing of QP runtime states, resulting in non-negligible overhead and prolonged profiling times. More recent works from industry like Mycroft [17] and Aegis [18] adopt a dependency-based tracing system to track network issues. However, they struggle to capture transient network dynamics. NCCL Profiler [12] and Inspector [13] can provide observability for each CC. However, both fail to reflect real-time network status, as multiple CCs may share one NIC simultaneously. Although sketches implemented in programmable switches [65] can monitor $O(\mu s)$ network dynamics, programmable switches are difficult to deploy in production-level clusters.

3 VCCL Design

3.1 Overview

Figure 3 demonstrates VCCL’s overview. Generally, VCCL works as a middleware between upper-layer LLM training frameworks [48, 52, 56] and underlying hardware. VCCL uses the unified programming interfaces (CUDA and RDMA

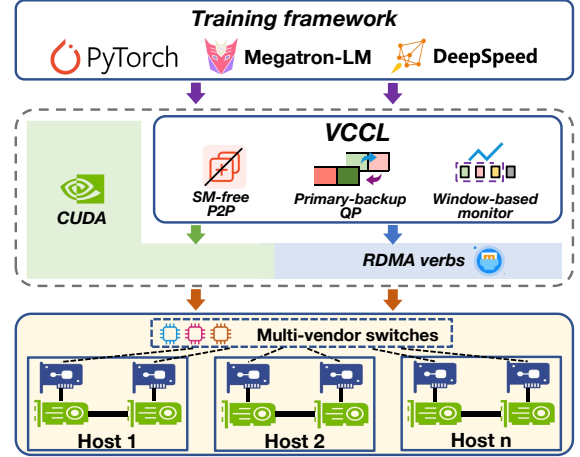


Figure 3: VCCL overview.

verbs) to enable CC among distributed GPUs during the training process. VCCL is developed based on NCCL, enabling the communication group with bootstrapping, topology search & graph construction and channel establishment. Additionally, VCCL introduces three extra modules: 1) SM-free P2P for resource-efficient and performant communication; 2) a primary-backup QP mechanism for fault tolerance. 3) a window-based monitor to capture network status and potential anomalies. These three modules collaborate to achieve an efficient, reliable and observable CCL.

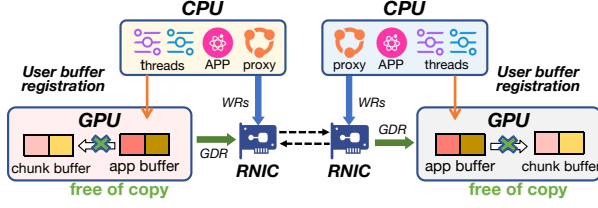
3.2 SM-free P2P

The contention for SMs between computation and communication (§2.2) constrains the potential for training efficiency. In contrast, CPUs typically exhibit lower utilization and greater resource availability compared to GPUs [1, 24, 28]. Consequently, offloading P2P operations from GPUs to CPUs seems a viable strategy. By eliminating GPU kernel launches, we can reclaim SM resources for general computation tasks, thereby accelerating the training process.

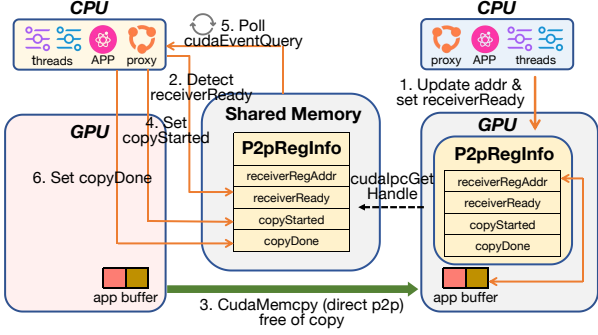
SM-free P2P in VCCL. The function of launched kernels in NCCL can be categorized into three aspects: (1) Intra-node data movement: Conducting intra-node communication or memory copies between application buffers and chunk buffers. (2) GPU-CPU synchronization: Coordinating the CPU-based control plane (Proxy) and the GPU-based data plane via shared pointer. (3) Dependency management on CUDA streams: Enforcing strict execution ordering among different streams. Given that training frameworks often pre-dispatch operations to enable asynchronous communication, inserting post-kernel events ensures that communication only starts after prerequisite computations finish and blocks subsequent dependent tasks. Here we detail how our SM-free P2P design effectively realizes the functionalities discussed above without relying on GPU kernels.

1) Intra-node data movement. For P2P communication be-

Due to page limit, we only draw inter-node SM-free P2P between ranks



(a) Inter-node SM-free P2P.



(b) Intra-node SM-free P2P.

Figure 4: SM-free P2P in VCCL.

tween ranks sharing the same local index or disable PXN [11], as depicted in Figure 4(a), where application and chunk buffers reside on the same GPU, VCCL leverages a zero-copy design inspired by NCCL’s User Buffer Registration feature [14]. When an upper-layer application initiates a SendRecv operation, VCCL first intercepts the memory registration request from the training framework. It then registers a user buffer via `ncclMemAlloc` and returns a handle, thereby enabling direct P2P data transmission from the application buffer and bypassing intermediate copies to the chunk buffer. Conversely, if PXN enabled, for communication between ranks with different local index, intra-node memory copies remain essential, since directly fetching data from application buffers by GDR incurs prohibitive PCIe overhead and potential cross-NUMA traversal. Therefore, we utilize `cudaMemcpy` on the sender side for data movement, relying on the GPU’s copy engines instead of launched kernels. The CPU Proxy then subsequently polls for completion via `cudaEventQuery` and initiates remote transmission upon `cudaSuccess`.

For intra-node P2P communication, as shown in Figure 4(b), we leverage a shared memory design. The receiver maintains a shared buffer, `P2pRegInfo`, containing specific four fields. Three of them—`receiverReady` (trigger data movement), `copyStarted` (confirm `cudaMemcpy` issuance), and `copyDone` (track operation completion)—are 1-bit flags to coordinate the sender and receiver. The fourth field, `receiverRegAddr`, is a dynamic pointer, pointing to the receiver’s application buffer to enable zero-copy transfer. During Connection phase, we utilize `cudaIpcGetHandle` to map this buffer into a shared region. During communication phase,

sharing the same local index.

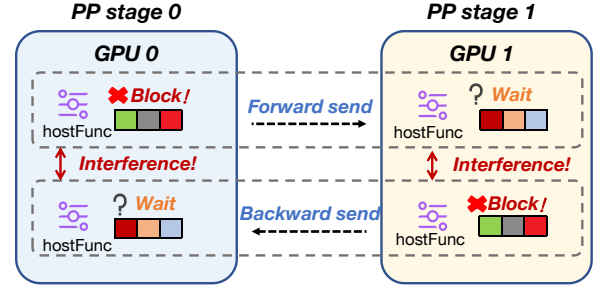


Figure 5: Potential hang with hostFunc.

the receiver first updates `receiverRegAddr` and changes `receiverReady` to 1 to notify the sender. Upon detection, the sender issues `cudaMemcpy`, sets `copyStarted` and subsequently polls for completion via `cudaEventQuery`. Finally, the sender sets `copyDone` to finalize the transmission.

2) GPU-CPU synchronization. In standard NCCL, coordination relies on GPU-CPU synchronization via shared volatile variables (e.g., `posted`). The CPU proxy must continuously poll these flags to detect the completion of kernel-side data movement before issuing subsequent operations, such as posting WRs. This polling mechanism inevitably incurs significant synchronization latency. By migrating data-plane control from GPU to CPU, VCCL effectively eliminates the overhead associated with GPU-CPU synchronization. For inter-node P2P, we utilize CPU-based memory copy operations to facilitate pipelined transmission. Similarly, for intra-node P2P, the aforementioned shared memory design is employed for state synchronization, ensuring that the process remains entirely free of GPU-CPU synchronization costs.

3) Dependency management on CUDA streams. To address the absence of kernels on the CUDA stream, we first utilize `cudaLaunchHostFunc` to manage stream execution order via the CPU. Specifically, VCCL replaces standard kernels with host functions to maintain dependency constraints. VCCL employs two synchronization primitives: `proxyReadyEvent` (to signal buffer readiness) and `proxyOpCount` (to track pending operations). The host function first awaits prerequisite computations and then asserts `proxyReadyEvent` to enable subsequent communication. It then polls `proxyOpCount` until it reaches zero, effectively blocking dependent tasks. However, we observe occasional hang during training with this SM-free design. After deep investigation, we find this issue stems from the single-thread limitation of `hostFunc`. As illustrated in Figure 5, a deadlock arises when the host function on GPU 0 blocks waiting for the `proxyReadyEvent` of backward communication from GPU 1. Simultaneously, GPU 1 awaits forward communication from GPU 0; however, GPU 0 cannot trigger the necessary ready signal because its host execution thread is blocked. To resolve this, we initially modify the training framework (e.g., Megatron [56]) to merge bidirectional P2P communication groups (GPU 0 ↔ GPU 1), successfully eliminating the hang.

However, this approach compromises deployability, as it

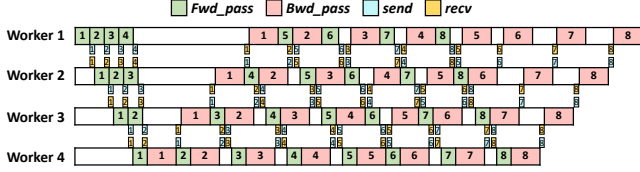
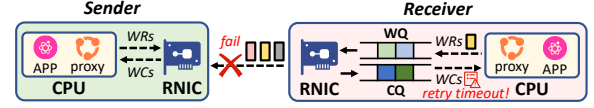


Figure 6: PP with VCCL.

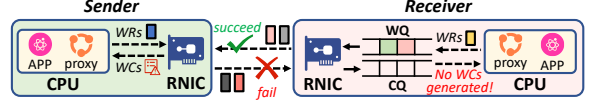
requires modifications to frameworks. We explore a fully non-intrusive alternative that preserves framework compatibility without requiring any code changes. We point out a key limitation of `cudaLaunchHostFunc` is that it is not a stream-native progress mechanism: it invokes CPU callbacks through CUDA’s internal host execution path, where callbacks from independent streams may be serialized. In contrast, `cuStreamWriteValue` and `cuStreamWaitValue` are stream memory operations enqueued directly into CUDA streams. They update or wait on memory locations in stream order without relying on host-callback execution, thereby avoiding serialization-induced deadlocks while preserving dependency constraints. By adopting the two stream memory operations, we devise a mechanism where `cuStreamWriteValue` updates the `proxyReadyEvent` and `cuStreamWaitValue` blocks the CUDA stream until `proxyOpCount` returns to zero. This approach guarantees strict execution ordering without modifying the training framework.

Improve overlaps in PP. Pipeline parallelism [56] uses P2P to synchronize the activations/gradients of forward/backward passes among GPU workers. However, P2P communication in NCCL has to compete for GPU SM resources with forward & backward passes, leading to compromised computation performance during overlap. In contrast, VCCL offloads P2P communication from GPUs to CPUs, and thus spares the SM resources for computation, empowering faster forward & backward passes and better overlaps to hide the P2P communication overhead.

Figure 6 shows VCCL’s PP progress using 1F1B strategy [41] with 8 microbatches and 4 GPU workers. Unlike kernel-based P2P that conducts computation and P2P communication in sequence, VCCL overlaps the P2P communication with forward & backward passes during the PP process. Generally, after finishing a forward/backward pass of a microbatch, VCCL allows the worker to proceed to the backward/forward pass of the next microbatch, and simultaneously send the activation/gradient to the next/previous worker. As a receiver, the worker can also `recv` the results from a sender, while performing a forward or backward pass at the same time. Compared to naive PP by NCCL, VCCL overlaps the P2P communication and enables faster forward & backward passes with more SM resources, therefore accelerating the entire LLM training.



(a) Case 1: triggered by local WC.



(b) Case 2: triggered by timeout threshold.

Figure 7: Perception of link failures.

3.3 Fault Tolerance with Backup QPs

VCCL adopts a primary-backup QP mechanism to tolerate link failures. Though previous works like Mooncake [50] propose a similar fail-safe solution, to the best of our knowledge, VCCL is the first work that successfully integrates such a feature into CCL with production validation. We mainly detail the underlying designs in this subsection.

Creation of Backup QPs. During the bootstrap for the inter-node connection, in addition to a primary QP that uses the closest RNIC, VCCL creates a backup QP by using the second-closest RNIC for each GPU. If a link fails due to port downs, VCCL migrates the traffic from the primary QP to the backup QP, and retransmits the data at the breakpoint. VCCL also monitors the failed link, and when it is fixed, VCCL turns back to the primary QP for faster transmission. Note that if the RNIC has dual ports, VCCL creates the backup QP on the other port of the same RNIC, enjoying the same hardware distance as the primary QP.

Failure perception. When an RNIC port down occurs, the sender cannot determine whether the inflight data has been received by the receiver. To address this, we employ a receiver-driven QP switching strategy, leveraging NCCL’s built-in CTS (clear to send) message mechanism to detect link failures. We define two trigger conditions for QP switching to handle different failure scenarios. Figure 7(a) illustrates the first case, where a receiver-to-sender communication failure prevents the receiver from notifying the sender via CTS. In this case, after exhausting a pre-defined number of retries and exceeding VCCL’s retry timeout threshold (determined by `ICCL_IB_TIMEOUT` and `ICCL_IB_RETRY_CNT`), the receiver RNIC generates a WC error. This error notifies the Proxy of the communication failure, allowing the control plane to detect the port-down event and trigger QP switching. Notably, we intentionally retain this retry window because many failures in our deployments are short-lived link flaps, about half of which recover within seconds.

The second case, illustrated in Figure 7(b), presents a more complex scenario: the RNIC port fails after the receiver successfully transmits a CTS message, but before the subsequent data transmission from the sender completes. In this case, while the sender eventually receives a Work Completion (WC) retry timeout error, the receiver remains unaware of this fail-

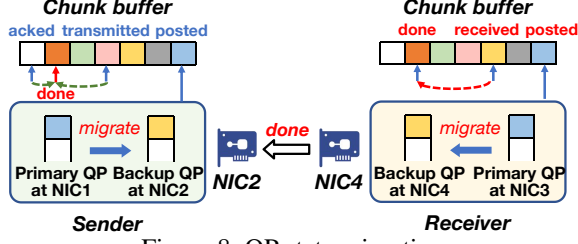


Figure 8: QP state migration.

ure and generates no local WC error to alert the control plane. To address this, we design a timeout-based trigger mechanism. When a WR is issued, the receiver records its timestamp and monitors the corresponding WC. If no WC is generated within the period δ , the receiver proactively resends a CTS message to probe the link state. If this probe fails, the receiver generates a local WC error, triggering the QP switching process as described in the first case. This “double-check” mechanism is critical; it prevents the system from misclassifying a healthy link as faulty when a sender is simply stalled due to upstream data dependencies—a common occurrence in collective communication processes. We set δ slightly larger than the retry-timeout threshold to account for queuing and propagation delays.

State synchronization and migration. To enable retransmission at the breakpoint (i.e., the last chunk failed to be sent), it is crucial to synchronize and migrate the states from the primary QP to the backup QP. As shown in Figure 8, VCCL maintains three pointers to track transmission and reception states at both the sender and receiver, along with a sender-side SyncFifo buffer for state synchronization (Table 2 in Appendix B). At the sender side, *posted* tracks chunks prepared by the GPU, *transmitted* marks chunks for which the CPU proxy has invoked `ibv_post_send`, and *acked* indicates the receipt of a WC acknowledged by the receiver. Correspondingly, the receiver tracks *posted* (ready for reception), *received* (invoked via `ibv_post_recv`), and *done* (data successfully committed to the application buffer). The receiver’s *done* pointer is synchronized with the sender’s *acked* pointer upon the completion of each chunk. Additionally, the SyncFifo stores the metadata required for in-place retransmission: `fifoHead` manages CTS synchronization, while `restartPos` and `error Port` allow the sender rank to identify the resumption point and the specific faulty link, respectively.

Upon perceiving a failure, the receiver actively retreats *received* to *done* to make backup QP start receiving data at the breakpoint. Then the receiver pushes requested objects to sender’s buffer by backup QP to ensure the consistent breakpoint at both sides. With mutual agreement of breakpoint location, the sender also retreats *acked* and *transmitted* to `restartPos`, notifying the Proxy to restart transmission from *acked* by backup QP.

Recovery of normal QPs. When RNIC port downs are fixed, VCCL migrates active transmissions from the backup QP

back to the primary QP. Because a link failure forces all associated primary QPs into the *error* state, a full state-machine reset is required. To bypass the significant overhead of QP re-establishment and bootstrap-based synchronization, we cache QP metadata (e.g., QPN) during the initial setup. We also observe that while the control plane latency for transitioning a QP (RESET → INIT → RTR → RTS) is negligible, the underlying hardware often requires a substantial warm-up period—often on the order of seconds—to become fully operational. To mask this latency, VCCL proactively initiates the QP reset sequence immediately upon failure perception, allowing hardware preparation to overlap with the failover period. Consequently, once the link is restored, VCCL can seamlessly leverage the previously described synchronization protocol to fallback to the primary QP.

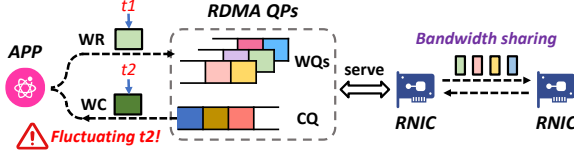
3.4 Online Network Performance Monitor

Although it is non-trivial to realize fine-grained network observability in current monitoring mechanism (§2.2), fortunately, the narrow-waist abstraction of RDMA programming [30, 32] can help us capture $O(\mu s)$ online inter-host network dynamics of each RNIC. Meanwhile, by modeling fine-grained online network bandwidth, we can also locate transient link anomalies for proactive isolation before they directly affect training or cause fatal failures, as Appendix K shows.

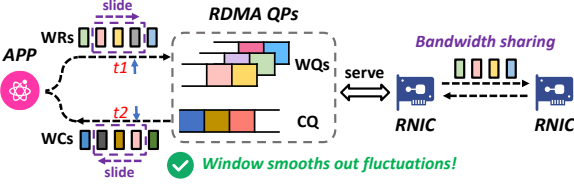
Capturing online network performance. To monitor fine-grained CC performance, a straightforward approach is to estimate per-message throughput from WR/WC timestamps, as shown in Figure 9(a). When the application posts a WR to a WQ, VCCL records timestamp t_1 ; when the corresponding WC is generated on the CQ, VCCL records t_2 . The throughput of message M is then estimated as $B = \frac{\omega(M)}{t_2 - t_1}$, where $\omega(M)$ is the message size. However, per-message estimation is unstable under concurrent traffic. When multiple messages share the same physical link, $t_2 - t_1$ captures not only transmission time, but also queuing delay and bandwidth interleaving with other requests. As a result, the estimated throughput becomes sensitive to transient cross-traffic rather than reflecting persistent changes in link behavior, making it unsuitable for robust anomaly detection.

VCCL instead uses a sliding window to estimate average throughput, as shown in Figure 9(b). For each window W , VCCL records t_1 as the posting time of the first WR in the window and t_2 as the completion time of the last corresponding WC. It then computes the window throughput as $\bar{B} = \frac{\sum_{i \in W} \omega(M_i)}{t_2 - t_1}$, where $i \in W$ indicates the i^{th} message inside the window W . By aggregating multiple messages, the window-based estimator amortizes transient queuing effects and provides a more stable signal for detecting sustained throughput degradation.

In practical usage, a proper window size is crucial to measuring accuracy. A window that is too small fails to suffi-



(a) Per-message scheme.



(b) Per-window scheme.

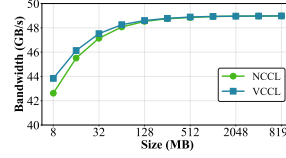
Figure 9: Bandwidth estimation.

ciently dampen transient noise, resulting in the same volatility observed in the naive per-message scheme. Conversely, an excessively large window suppresses short-term variations but may smooths out subtle but significant throughput shifts caused by actual network anomalies and obscuring transient dynamics. Therefore, developers should optimize the size of sliding window according to their cluster environment and network dynamics.

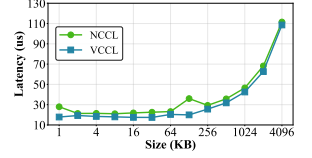
Network anomaly pinpointing. Beyond providing $O(\mu s)$ -level network monitoring, VCCL can effectively identify network anomalies by leveraging the window-based monitor. Given the inherent performance variance across different CC primitives, we employ an operation-aware heuristic. Specifically, VCCL compares the current bandwidth with the average preceding bandwidth (e.g., 10 ms). To mitigate false positives, an anomaly is identified only when two conditions are met simultaneously: (i) the current bandwidth drops by more than 50% relative to the previous average of the same primitive, and (ii) the remaining unsent data on the NIC (tracked via RDMA WR/WC lifecycle) exceeds twice the maximum value observed within the historical data. This dual-threshold strategy ensures precise localization of port-level degradation (see §4.3 for details). Notably, thresholds are empirically chosen to avoid false positives from cases like overlapping communication, following prior works [17].

4 Deployment: VCCL in production

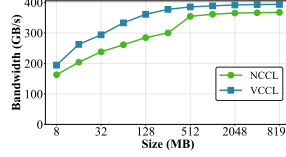
VCCL has been built on multiple versions of NCCL, with about additional 7,000 lines of C codes, which is publicly available at Github [8]. VCCL was initially deployed in a 2k-GPU cluster in December 2023, and has since been integrated into all our subsequent delivery clusters, scaling up to 24k GPUs. To protect our customer’s privacy, specific institution identities are omitted. Originally, VCCL was developed for customized optimizations, like topology-aware channel reconstruction or enhanced NIC selection in dual-port environments. Since NCCL has introduced some similar capabilities



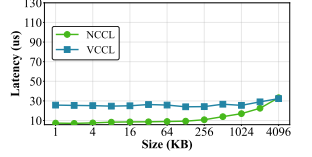
(a) Inter-node P2P bandwidth.



(b) Inter-node P2P latency.



(c) Intra-node P2P bandwidth.



(d) Intra-node P2P latency.

Figure 10: Bandwidth and latency performance.

during evolution, we only focus on the deployment of VCCL regarding the newly proposed features.

Cluster environment. To ensure the reproducibility of our experimental results, we describe an operational 24k-GPU cluster composed of multiple 3–4k-GPU sub-clusters. It includes: 1) 24k NVIDIA Hopper GPUs; 2) NVIDIA ConnectX-7 RNICs [43] and BlueField-3 DPUs [44]. Each server is equipped with 8 homogeneous GPUs and 9 RNICs. Each sub-cluster adopts a 1:1 oversubscribed two-layer rail-optimized network topology with 400Gbps bandwidth, and all sub-clusters are jointly managed via Kubernetes-based scheduling [38] as a unified production cluster.

4.1 Efficiency

By achieving perfect overlap between computation and communication, VCCL accelerates training progress, saving great GPU hours. Due to confidential reasons, the statistics showed below are all collected by task from our internal team.

P2P performance. We first use NCCL-Tests to demonstrate the P2P performance difference between VCCL and NCCL v2.26 [10]. To ensure a fair comparison, we explicitly implement the zero-copy mechanism for the NCCL baseline. Experimental data is depicted in Figure 10. For inter-node P2P, since VCCL and NCCL are both free of cumbersome buffer copies, they achieve similar bandwidth for large messages (> 8MB). Meanwhile, VCCL reduces small-message latency by 18.9% on average by eliminating GPU-CPU synchronization. For intra-node P2P, VCCL improves large-message bandwidth by approximately 7%. The reason behind is that its copy-engine-based approach issues wider transactions that better saturate NVLink compared to SM-based transfers. However, we also observe higher latency for small intra-node messages—a result of contention for limited copy-engine resources—this is not a critical bottleneck in production environments. In PP, message sizes typically exceed 32MB (explained in Appendix C), rendering small-message latency overhead negligible. Furthermore, the primary value of VCCL’s SM-free mechanism lies in eliminating SM contention, which maximizes the potential for computation-communication overlap. Consequently,

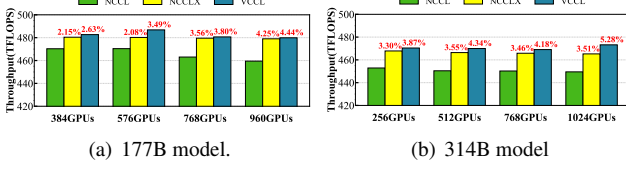


Figure 11: Training throughput.

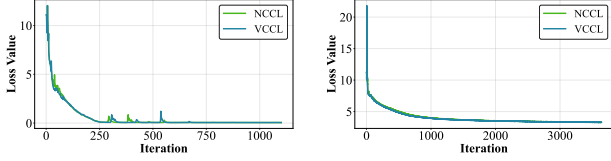


Figure 12: Loss values.

the bandwidth efficiency and SM availability gains far outweigh the minor latency penalties for small P2P messages.

Training performance. We then collect statistics from our internal team’s training tasks to evaluate the training performance of VCCL.

1) **Scalability.** We first evaluate the strong-scaling performance of VCCL against NCCL v2.26 and an NCCLX-like baseline that reproduces NCCLX’s key mechanism (i.e., launching a lightweight GPU kernel with only 1 SM to enforce stream ordering), but implemented in our codebase for integration and measurement consistency. Typically, the training workload is GPT-2 [51], an open-source dense LLM with different sizes of 177B and 314B. All our training tasks use Megatron-LM [56], with training hyperparameters and default settings listed in Appendix D. As shown in Figure 11, VCCL consistently outperforms NCCL across all settings, delivering an average training TFLOPS improvement of 4.00% and a maximum gain of up to 5.28%. Additionally, the NCCLX-based baseline exhibits performance degradations of up to 1.73% compared to VCCL, indicating that even the consumption of a single SM can measurably reduce training efficiency. At scale, such efficiency losses translate into substantial GPU time wastage during large-scale training. We explain the 5% training throughput gain in Appendix E.

2) **Model convergence.** To verify that the SM-free design of VCCL does not degrade convergence speed, we conduct extensive training of GPT-2 with 32B and 70B parameters. Figure 12 shows that VCCL exhibits the same loss value trend as NCCL for GPT-2 in different sizes, proving that VCCL’s SM-free P2P design using `waitValue` guarantees the correct interaction and order between computation and communication as kernels do in NCCL.

Resource consumption. We profile the SM and CPU resources consumption of VCCL and NCCL, indicating that VCCL successfully eliminates GPU kernel launches for communication, while introducing only about 2% additional CPU utilization (details can be seen in Appendix F).

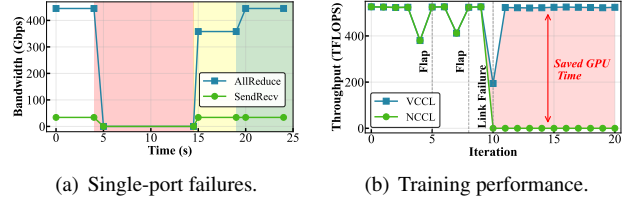


Figure 13: Tolerance to link failures.

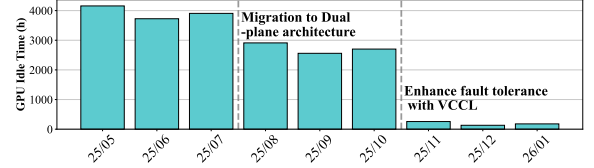


Figure 14: GPU idle time caused by link failures.

4.2 Reliability

Tolerance to link failures. To evaluate the reliability of VCCL, we first conduct NCCL-Tests of two representative communication workloads: `SendRecv` (primitive for pipeline parallelism) and `AllReduce` (widely used in tensor and data parallelism). Figure 13(a) depicts the runtime bandwidth of VCCL under a link failure. Specifically, we manually bring down a RNIC port from 4s to 19s, whose failed duration exceeds the default retry timeout, and bring up the RNIC port at 19s again. The red area denotes the retry period, and yellow area denotes VCCL is transmitting using the backup QP, while green area denotes VCCL switches back to the primary QP when the RNIC port is up again. When a port down occurs, VCCL spends about 10s to allow hardware retransmission on the same port, staying at 0GB/s during the retry period. After the retry window, VCCL switches to the backup QP using an alternative healthy RNIC port, ensuring in-place communication recovery. Additionally, when the RNIC port is brought up again, VCCL can switch back to the primary QP and deliver the ideal bandwidth. Meanwhile, to evaluate the resilience of VCCL against multi-port failures, we perform an `AllReduce` stress test, where detailed results are provided in Appendix G.

In addition, we select a real-world 70B-parameter training task to demonstrate VCCL’s robustness under practical failure conditions. As is depicted in Figure 13(b), while both systems experience temporary stalls during minor link flaps, NCCL inevitably hangs when facing severe link failures that exceed hardware retransmission limits. In contrast, VCCL leverages its primary-backup QP mechanism to recover training after a timeout, with TFLOPS almost remaining constant. The red region represents the significant GPU hours VCCL saves. In large-scale production deployments, where link failures requiring manual intervention are increasingly frequent, VCCL’s ability to maintain continuity offers substantial savings in computational resources.

The necessity of link failure tolerance. We share the impact of link failures within a production cluster of 24K GPUs in

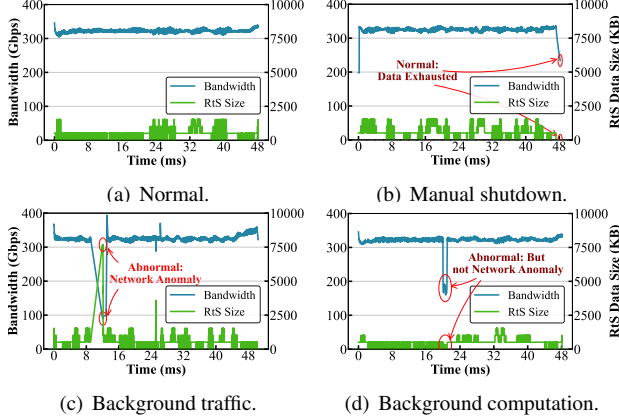


Figure 15: Network performance tracing of CC across four representative cases.

Figure 14. Prior to August 2025, the cluster utilize a single-plane topology where link failures resulted in massive GPU-hour wastage. Although some dual-port NICs were migrated to a dual-plane deployment, reducing about 29.6% idle GPU time, standard NIC bonding is inherently inapplicable to single-plane architecture, which still hosts the vast majority of GPUs in the cluster. The subsequent deployment of VCCL’s fault tolerance feature in October successfully addressed this issue, leading to a nearly 90% reduction in idle GPU resources. This substantial decrease in computational overhead underscores the critical necessity of runtime link-failure tolerance in large-scale distributed training clusters.

System overhead. Although VCCL maintains twice as many QPs, active QP numbers are unchanged, so RNIC on-chip SRAM pressure does not increase. Meanwhile, in a 1024-GPU setting with TP=4, PP=4, and 16 channels, each RNIC needs only 192 local connections, far below modern RNIC capacity, e.g., ConnectX-7 supports 131,072 QPs.

4.3 Observability

Case study: Pinpointing network straggler. Building on the $O(\mu s)$ monitor (due to space limit, we detail the wisdom of sliding-window based monitor in Appendix H), VCCL enables precise localization of network stragglers. Figure 15 evaluates the effectiveness of VCCL’s heuristic pinpointing algorithm under four representative scenarios: (1) a normal CC task, (2) the manual termination of a CC task, (3) a CC task under network interference (small-packet perftest), and (4) a CC task under transient hardware interference (GPU-burn). As shown in the results, the normal case (case 1) maintains stable bandwidth and remaining-to-send (RTS), indicating a healthy state. In case 2, VCCL correctly classifies the terminal bandwidth decline as normal behavior, attributing it to the exhaustion of the NIC’s data buffer rather than a network anomaly. Crucially, VCCL distinguishes between network and non-network issues. In case 3, the simultaneous drop in bandwidth and RTS accumulation allows VCCL to

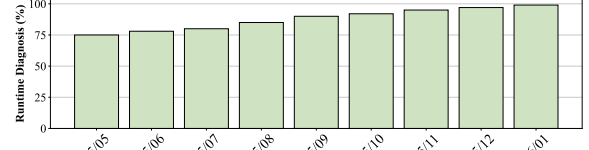


Figure 16: Runtime diagnosis percentage.

pinpoint a network anomaly. By contrast, in case 4, despite the significant bandwidth drop caused by GPU resource contention, the lack of data accumulation on the NIC prevents a false positive. These results underscore VCCL’s ability to accurately localize network stragglers while remaining robust to other performance fluctuations.

Full-stack troubleshooting with VCCL. To enable effective runtime diagnosis, we propose a full-stack troubleshooting system with VCCL designed for fault localization and recovery, the details of which can be seen in Appendix I. We share the runtime diagnosis percentage in one of our operating cluster. As depicted in Figure 16, with the integration of VCCL for network straggler pinpointing, our full-stack troubleshooting platform completes its final missing component and drives the runtime diagnosis percentage gradually approaching 100%. Given the high GPU wastage of offline diagnosis, these real-world results demonstrate the effectiveness of our full-stack troubleshooting platform.

System overhead. We evaluate the system overhead of our window-based monitor in terms of CPU and memory. As shown in Table 5 in Appendix F, it incurs only moderate CPU overhead and negligible memory footprint.

4.4 Other System-level optimizations

Optimizing memory usage. During the deployment of VCCL, we observe unexpected high GPU memory consumption, reaching 10GB HBM for specific MoE models. This inefficiency, which severely constrains model parallelism strategies, stems from NCCL’s default aggressive pre-allocation across all protocols and channels, as well as its reliance on intermediate GPU buffers for intra-node data movement. To mitigate this, VCCL introduces a dynamic memory pool design. First, VCCL adopts a lazy allocation strategy, where resources are assigned to connections only upon their first runtime usage. The system maintains a 2MB-aligned memory pool to allocate minimal necessary memory for each peer, dynamically expanding via `cuCallocAsync` when the pool is exhausted. Second, by leveraging zero-copy mechanisms for both intra-node and inter-node P2P primitives, we are allowed to eliminate intermediate buffers, thereby minimizing the memory footprint. Our experimental data shows that VCCL achieves up to 26.7% memory reduction in practical deployments of models with complex parallelism. More details can be found in Appendix J.

Traffic imbalance in bonding environment. When enabling VCCL in dual-plane clusters with bonding, we encounter severe traffic load imbalance. Specifically, in dual-port environ-

ments without bonding enabled, a two-node ReduceScatter benchmark in NCCL-Tests achieves up to 380 Gbps bandwidth, whereas enabling bonding reduces the bandwidth to approximately 220 Gbps. To mitigate this, after deep investigation of Mellanox driver internals, we utilize `mlx5dv_modify_qp_lag_port` during the initialization phase to evenly distribute QPs across the two physical ports. This approach successfully enables the bonding environment to achieve bandwidth performance similar to that of a single-port one.

5 Experience and lessons

In this section, we share key experiences and lessons we learned during the deployment of VCCL (additional experience can be found in Appendix K).

Performance gain of SM-free. While §4.1 demonstrates the significant performance gain of VCCL’s SM-free design, we encounter several unexpected behaviors in deployment. First, occasionally, we observe that VCCL exhibits little or no advantage over NCCL. Further investigation reveals that this phenomenon stems from a small batch size that leads to underutilization of GPU, allowing NCCL to also overlap communication in PP and thus masking the benefit of saving SM resources. After we increase the batch size to fully saturate the GPU, the issue is resolved. Besides, we find that relative throughput gain of VCCL gradually decreases as the training cluster scales out. After careful investigation, we point out this is not caused by scalability limitations of VCCL, but by growing DP communication overhead. The performance benefit can be modeled as: $I = \frac{\frac{C}{T_v + \alpha} - \frac{C}{T_n + \alpha}}{\frac{C}{T_n + \alpha}} = \frac{T_n - T_v}{T_v + \alpha}$, where C represents TFLOPS per GPU per iteration, T_v and T_n denote per-iteration compute time under VCCL and NCCL, respectively, and α represents DP communication overhead. Specifically, since the communication pattern within DP group follows the ring algorithm over a single-rail interconnect, communication overhead of AllReduce or ReduceScatter & AllGather exhibits linear scaling, causing I to decrease with cluster size. Despite this, even though the relative TFLOPS gain decreases at larger scale, the increased GPU count preserves significant absolute GPU time savings from VCCL’s SM-free design.

Misleading cases without troubleshooting. Without a full-stack troubleshooting platform, heterogeneous root causes frequently collapse into indistinguishable CC-level exceptions, making root-cause pinpointing ambiguous. We present two typical cases that lead to CC throughput degradation. 1) *Case 1: insufficient CPU cores.* In this case, CC throughput only achieved 30% of ideal bandwidth. Subsequent investigation reveals that this issue stems from the misconfiguration of setting an upper limit of at most 8 CPU cores per developer, in which case VCCL’s network proxy did not have enough CPU resources to conduct communication. However, by using `lscpu` on the platform, we can indeed easily pin-

point the root cause. 2) *Case 2: misconfiguration of GPU fan speed.* We encountered intermittent drops during LLM training, and Nsight [47] analysis showed that other metrics excluding communication worked well as usual. Further investigation identifies a fan speed limit misconfiguration on a single GPU, such that it was overheated and slowed down the communication, resulting in compromised CC throughput. In fact, this misleading case can also be effectively diagnosed through hardware-level inspection with `ipmitool`.

Lessons we learned as a CCL provider. In some specific cases, our customers encountered failures that the troubleshooting platform reported no hardware or software anomalies. However, they observed these failures could be eliminated simply by switching from VCCL to NCCL, which naturally leads to the perception that the issues were caused by VCCL. Through careful investigation, we find that these failures are often not inherent bugs of VCCL, but rather stem from subtle deployment issues. Below, we present two representative lessons we learned in real-world deployments.

1) *Environment variable misconfigurations can cause fatal failures.* After deploying VCCL, some customers experienced severe segmentation faults. Notably, these failures disappeared when reverting to NCCL, making the issue particularly misleading. We initially analyzed the core dumps using `gdb`, yet no obvious software defects or memory violations attributable to VCCL were identified. A deeper inspection of the training scripts eventually revealed the root cause: the environment variable `ICCL_NET_PLUGIN` was mistakenly set, forcing VCCL to load NCCL’s default network plugin. This resulted in conflicting definitions of internal data structures between VCCL and NCCL, ultimately causing undefined behavior and crashes. Once this environment variable was unset, VCCL operated correctly. This case highlights that, as a CCL provider, careful recommendation of environment variables is essential. Even seemingly small configuration errors can introduce subtle and catastrophic failures that are difficult to diagnose through conventional debugging tools.

2) *Rule out NCCL bugs before attributing failures to VCCL.* We encountered a more subtle failure scenario in which VCCL consistently crashed while NCCL functioned correctly. This difference initially suggested a defect in VCCL. Further investigation revealed that the issue was related to MTU inconsistency. At the time, VCCL was based on NCCL v2.21.5, which did not yet include software-side mitigations for handling MTU mismatches. In contrast, the NCCL plugin bundled in the customer’s runtime environment was NCCL v2.24, where this issue had already been fixed. As a result, NCCL appeared to function correctly, while VCCL, built on the older version, inherited the bug and failed. This experience suggests that, when diagnosing suspected failures of VCCL, it is critical to first align the NCCL versions used by both VCCL and the runtime environment. Excluding NCCL-induced bugs early in the debugging process helps avoid misattribution of faults and allows more effective improvement of VCCL’s

robustness and performance.

Robust networking configuration. Networking configuration is critical to VCCL’s reliability, yet we encounter a subtle but fatal failure—unexpected changes of Global Identifier (GID). In RoCEv2 environments, while GIDs should remain static after initialization, we discover that default OS-level network managers (e.g., `NetworkManager`) trigger a GID increment upon RNIC port flaps even if the original IP is reclaimed. This non-deterministic GID shift inevitably collapses the communication collective and the LLM training job. While driver-level patches can suppress GID increments, they introduce compatibility risks in virtualized networks. Consequently, we explore an alternative by changing the configuration method for IP addresses in a lightweight and reliable manner. We take Linux Ubuntu operating system as an example. Ubuntu has two basic IP configuration methods, i.e., `NetworkManager` and `Networking` service. By `NetworkManager`, GIDs can change because of a temporary loss of IP addresses when RNIC port are down, illustrated in the aforementioned case. Instead, when we use `Networking` to configure the IP addresses, even if port down happens, its IP address still remains unchanged and avoids a temporary loss, leading to an unchanged GID number. Other Linux distributions can also use the similar approaches for IP address configuration.

6 Discussion

SM-free and device-initiated. Recent industry advancements [7, 33] leverage device-initiated APIs (e.g., `GPUDirect Async` [16]) to minimize network latency by offloading control plane operations from CPU proxy to GPU kernels. While this approach is fundamentally distinct from VCCL’s SM-free architecture, we argue that the two paradigms are complementary. Device-initiated APIs accelerate RNIC doorbell triggering, offering significant latency optimization for small messages, which is a critical factor for latency-sensitive scenarios like distributed LLM inference. Conversely, in training scenarios where efficiency hinges on maximizing overlapping between computation and communication rather than pure latency, VCCL’s SM-free design is superior.

SM-free for other reduction-free primitives. In this paper, we mainly focus on optimizations of P2P primitives. The reasons behind this choice is that VCCL’s SM-free P2P implementation enables perfect overlapping of computation and communication within the 1F1B pipeline, thereby optimizing the critical path of LLM training. We believe the SM-free philosophy of VCCL can also be extended to other reduction-free CC primitives. Specifically, for `AlltoAll` in Mixture-of-Experts (MoE) workloads, our SM-free design can also facilitate overlapping computation with the dispatch and combine phases. However, optimizing `AlltoAll` entails intricate traffic scheduling considerations to avoid many-to-one incast. On the other hand, `AllGather` often lies on the critical path of training and lacks sufficient computation-communication

overlap opportunities. Therefore, we leave the extension to other primitives as our future work.

Limitation of fault tolerance. While VCCL provides robust tolerance against multi-port link failures, it remains vulnerable to the simultaneous failure of both the primary and backup RNICs due to our fixed QP placement strategy. Fortunately, our production-level observations indicate that the probability of such correlated failures is near zero, justifying our current design choice. Nonetheless, we believe addressing this corner case through more flexible backup mechanisms offers a compelling direction for academic research. Furthermore, given the significant bandwidth fluctuations observed during multi-port failures, developing adaptive communication scheduling based on runtime hardware resource availability represents another promising area for future exploration.

7 Related Work

CC Optimization. Besides NCCL and various xCCLs from industrial community, many works from academia are proposed to optimize CC performance. Blink [59], SCCL [3], TACCL [54] and SyCCL [4] synthesize collective algorithms according to heterogeneous topology of intra-host and inter-host. TCCL [31], TECCL [37], MCCS [61], AutoCCL [62], ResCCL [36] and Wang et al. [64] optimize the CC scheduling and configuration by topology, traffic workload and user information. Despite promising, these works are not mature and comprehensive enough to address our challenges in production-level clusters.

Production-level LLM training. ByteDance [29], Meta [19, 21, 39, 57], Alibaba [18, 49], DeepSeek [1, 23], Kuaishou [63] and Shanghai AI lab [24] share their practice on production-level LLM training, including optimizations on efficiency of computation & communication, reliability and observability for their large-scale GPU training clusters. However, they do not provide a systematic prospective on requirements, designs and deployment of collective communications. We expect VCCL together with existing works can contribute to a more thriving LLM training ecosystem.

8 Conclusion

In this paper, we design VCCL, an efficient, reliable and observable CCL with SM-free P2P, primary-backup QPs and window-based network monitor for production-level GPU clusters. Our production deployment and evaluation show that VCCL improves training performance by saving SM resources, maintains training under RNIC port failures, and detects $O(\mu s)$ network anomalies. We also share operational lessons from running VCCL at scale, and hope they motivate the community to revisit CC and build better CCL service for their clusters.

References

- [1] Wei An, Xiao Bi, Guanting Chen, Shanhuang Chen, Chengqi Deng, Honghui Ding, Kai Dong, Qiushi Du, Wenjun Gao, Kang Guan, Jianzhong Guo, Yongqiang Guo, Zhe Fu, Ying He, Panpan Huang, Jiashi Li, Wenfeng Liang, Xiaodong Liu, Xin Liu, Yiyuan Liu, Yuxuan Liu, Shanghao Lu, Xuan Lu, Xiaotao Nie, Tian Pei, Junjie Qiu, Hui Qu, Zehui Ren, Zhangli Sha, Xuecheng Su, Xiaowen Sun, Yixuan Tan, Minghui Tang, Shiyu Wang, Yaohui Wang, Yongji Wang, Ziwei Xie, Yiliang Xiong, Yanhong Xu, Shengfeng Ye, Shuiping Yu, Yukun Zha, Liyue Zhang, Haowei Zhang, Mingchuan Zhang, Wentao Zhang, Yichao Zhang, Chenggang Zhao, Yao Zhao, Shangyan Zhou, Shunfeng Zhou, and Yuheng Zou. Fire-flyer ai-hpc: A cost-effective software-hardware co-design for deep learning. In *SC*, 2024.
- [2] Wei Bai, Shanim Sainul Abdeen, Ankit Agrawal, Krishan Kumar Attre, Paramvir Bahl, Ameya Bhagat, Gowri Bhaskara, Tanya Brokhman, Lei Cao, et al. Empowering azure storage with rdma. In *NSDI*, 2023.
- [3] Zixian Cai, Zhengyang Liu, Saeed Maleki, Madanlal Musuvathi, Todd Mytkowicz, Jacob Nelson, and Olli Saarikivi. Synthesizing optimal collective algorithms. In *PPoPP*, 2021.
- [4] Jiamin Cao, Shangfeng Shi, Jiaqi Gao, Weisen Liu, Yifan Yang, Yichi Xu, Zhilong Zheng, Yu Guan, Kun Qian, Ying Liu, et al. Syccol: Exploiting symmetry for efficient collective communication scheduling. In *SIGCOMM*, 2025.
- [5] Chang Chen, Xiuhong Li, Qianchao Zhu, Jiangfei Duan, Peng Sun, Xingcheng Zhang, and Chao Yang. Centauri: Enabling efficient scheduling for communication-computation overlap in large model training via communication partitioning. In *ASPLOS*, 2024.
- [6] Ziteng Chen, Menghao Zhang, Guanyu Li, Jiahao Cao, Yang Jing, Mingwei Xu, Renjie Xie, He Liu, Fangzheng Jiao, and Xiaohe Hu. Paraleon: Automatic and adaptive tuning for dcqn parameters in rdma networks. In *ICNP*, 2024.
- [7] DeepEP contributors. DeepEP, 2025. <https://github.com/deepseek-ai/DeepEP>.
- [8] ICCL Contributors. Iccl source code, 2025. <https://github.com/sii-research/VCCL>.
- [9] Linux-RDMA contributors. Rdma core userspace libraries and daemons, 2025. <https://github.com/linux-rdma/rdma-core>.
- [10] NCCL Contributors. Nvidia collective communications library (nccl), 2016. <https://github.com/NVIDIA/nccl>.
- [11] NCCL contributors. Nccl pxn, 2022. <https://developer.nvidia.com/blog/doubling-all2all-performance-with-nvidia-collective-communication-library-2-12/>.
- [12] NCCL contributors. Nccl environment variables, 2025. <https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/env.html>.
- [13] NCCL contributors. Nccl inspector, 2025. <https://developer.nvidia.com/blog/enhancing-communication-observability-of-ai-workloads-with-nccl-inspector/>.
- [14] NCCL contributors. Nccl user buffer registration, 2025. <https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/usage/bufferreg.html>.
- [15] NCCL Contributors. Nccl v2.29.2-1 release, 2026. <https://github.com/NVIDIA/nccl/commit/ebd1e929285881cb4df02c3459588a2e12b2d8c0>.
- [16] NVSHMEM contributors. Gpudirect async, 2022. <https://developer.nvidia.com/blog/improving-network-performance-of-hpc-systems-using-nvidia-magnum-io-nvshmem-and-gpudirect-async/>.
- [17] Yangtao Deng, Lei Zhang, Qinlong Wang, Xiaoyun Zhi, Xinlei Zhang, Zhuo Jiang, Haohan Xu, Lei Wang, Zuquan Song, Gaohong Liu, et al. Mycroft: Tracing dependencies in collective communication towards reliable llm training. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, pages 254–269, 2025.
- [18] Jianbo Dong, Kun Qian, Pengcheng Zhang, Zhilong Zheng, Liang Chen, Fei Feng, Yichi Xu, Yikai Zhu, Gang Lu, Xue Li, et al. Evolution of aegis: Fault diagnosis for {AI} model training service in production. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, pages 865–881, 2025.
- [19] Adithya Gangidi, Rui Miao, Shengbao Zheng, Sai Jayesh Bondu, Guilherme Goes, Hany Morsy, Rohit Puri, Mohammad Riftadi, Ashmitha Jeevaraj Shetty, Jingyi Yang, et al. Rdma over ethernet for distributed training at meta scale. In *SIGCOMM*, 2024.
- [20] Clinton Gormley and Zachary Tong. *Elasticsearch: the definitive guide: a distributed real-time search and analytics engine*. " O'Reilly Media, Inc.", 2015.

- [21] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [22] Chuanxiong Guo, Haitao Wu, Zhong Deng, Gaurav Soni, Jianxi Ye, Jitu Padhye, and Marina Lipshteyn. Rdma over commodity ethernet at scale. In *SIGCOMM*, 2016.
- [23] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [24] Qinghao Hu, Zhisheng Ye, Zerui Wang, Guoteng Wang, Meng Zhang, Qiaoling Chen, Peng Sun, Dahua Lin, Xiaolin Wang, Yingwei Luo, et al. Characterization of large language model development in the datacenter. In *NSDI*, 2024.
- [25] IEEE. IEEE 802.1 Qbb - Priority-based Flow Control. <https://1.ieee802.org/dcb/802-1qbb/>, 2011.
- [26] IETF. Snmp, 1988. <https://www.snmp.com/protocol/>.
- [27] Abhinav Jangda, Jun Huang, Guodong Liu, Amir Hossein Nodehi Sabet, Saeed Maleki, Youshan Miao, Madanlal Musuvathi, Todd Mytkowicz, and Olli Saarikivi. Breaking the computation and communication abstraction barrier in distributed machine learning workloads. In *ASPLOS*, 2022.
- [28] Yimin Jiang, Yibo Zhu, Chang Lan, Bairen Yi, Yong Cui, and Chuanxiong Guo. A unified architecture for accelerating distributed dnn training in heterogeneous gpu/cpu clusters. In *OSDI*, 2020.
- [29] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, et al. Megascale: Scaling large language model training to more than 10,000 gpus. In *NSDI*, 2024.
- [30] Daehyeok Kim, Tianlong Yu, Hongqiang Harry Liu, Yibo Zhu, Jitu Padhye, Shachar Raindel, Chuanxiong Guo, Vyas Sekar, and Srinivasan Seshan. Freeflow: Software-based virtual rdma networking for containerized clouds. In *NSDI*, 2019.
- [31] Heehoon Kim, Junyeol Ryu, and Jaejin Lee. Tccl: Discovering better communication paths for pcie gpu clusters. In *ASPLOS*, 2024.
- [32] Xinhao Kong, Yibo Zhu, Huaping Zhou, Zhuo Jiang, Jianxi Ye, Chuanxiong Guo, and Danyang Zhuo. Collie: Finding performance anomalies in rdma subsystems. In *NSDI*, 2022.
- [33] Nandor Licker, Kevin Hu, Vladimir Zaytsev, and Lequn Chen. pplx-kernels: Perplexity MoE kernels. <https://github.com/perplexityai/pplx-kernels>, 2025.
- [34] Kefei Liu, Zhuo Jiang, Jiao Zhang, Shixian Guo, Xuan Zhang, Yangyang Bai, Yongbin Dong, Feng Luo, Zhang Zhang, Lei Wang, et al. R-pingmesh: A service-aware roce network monitoring and diagnostic system. In *SIGCOMM*, 2024.
- [35] Kefei Liu, Zhuo Jiang, Jiao Zhang, Haoran Wei, Xiaolong Zhong, Lizhuang Tan, Tian Pan, and Tao Huang. Hostping: Diagnosing intra-host network bottlenecks in rdma servers. In *NSDI*, 2023.
- [36] Tongrui Liu, Chenyang Hei, Fuliang Li, Chengxi Gao, Jiamin Cao, Tianshu Wang, Ennan Zhai, and Xingwei Wang. Resccl: Resource-efficient scheduling for collective communication. In *SIGCOMM*, 2025.
- [37] Xuting Liu, Behnaz Arzani, Siva Kesava Reddy Kakarla, Liangyu Zhao, Vincent Liu, Miguel Castro, Srikanth Kandula, and Luke Marshall. Rethinking machine learning collective communication as a multi-commodity flow problem. In *SIGCOMM*, 2024.
- [38] Marko Luksa. *Kubernetes in action*. Simon and Schuster, 2017.
- [39] Kiran Kumar Matam, Hani Ramezani, Fan Wang, Zeliang Chen, Yue Dong, Maomao Ding, Zhiwei Zhao, Zhengyu Zhang, Ellie Wen, and Assaf Eisenman. Quick-update: a real-time personalization system for large-scale recommendation models. In *NSDI*, 2024.
- [40] Microsoft. Msccl: Microsoft collective communication library, 2022. <https://github.com/microsoft/msccl>.
- [41] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R Devanur, Gregory R Ganger, Phillip B Gibbons, and Matei Zaharia. Pipedream: Generalized pipeline parallelism for dnn training. In *SOSP*, 2019.
- [42] NVIDIA. Nccl-tests, 2016. <https://github.com/NVIDIA/nccl-tests>.
- [43] NVIDIA. Nvidia connectx-7 datasheet, 2022. <https://www.nvidia.com/content/dam/en-zz/Solutions/networking/ethernet-adapters/connectx-7-datasheet-Final.pdf>.
- [44] NVIDIA. Nvidia bluefield-3 dpu, 2025.

- [45] NVIDIA. Nvidia cuda toolkit, 2025. <https://developer.nvidia.com/cuda-toolkit>.
- [46] NVIDIA. Nvidia dcgm, 2025. <https://developer.nvidia.com/dcgm>.
- [47] NVIDIA. Nvidia nsight systems, 2025. <https://developer.nvidia.com/nsight-systems>.
- [48] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In *NeurIPS*, 2019.
- [49] Kun Qian, Yongqing Xi, Jiamin Cao, Jiaqi Gao, Yichi Xu, Yu Guan, Binzhang Fu, Xuemei Shi, Fangbo Zhu, Rui Miao, Chao Wang, Peng Wang, Pengcheng Zhang, Xianlong Zeng, Zhiping Yao, Ennan Zhai, and Dennis Cai. Alibaba hpn: A data center network for large language model training. In *SIGCOMM*, 2024.
- [50] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading more storage for less computation—a {KVCache-centric} architecture for serving {LLM} chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, pages 155–170, 2025.
- [51] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. 2019.
- [52] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. pages 3505–3506, 2020.
- [53] ROCm. Rocm communication collectives library (rccl), 2025. <https://github.com/ROCm/rccl>.
- [54] Aashaka Shah, Vijay Chidambaram, Meghan Cowan, Saeed Maleki, Madan Musuvathi, Todd Mytkowicz, Jacob Nelson, Olli Saarikivi, and Rachee Singh. Taccl: Guiding collective algorithm synthesis using communication sketches. In *NSDI*, 2023.
- [55] Rong Shi, Sreeram Potluri, Khaled Hamidouche, Jonathan Perkins, Mingzhe Li, Davide Rossetti, and Dhabaleswar K DK Panda. Designing efficient small message transfer mechanism for inter-node mpi communication on infiniband gpu clusters. In *2014 21st International Conference on High Performance Computing (HiPC)*, pages 1–10. IEEE, 2014.
- [56] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- [57] Min Si, Pavan Balaji, Yongzhou Chen, Ching-Hsiang Chu, Adi Gangidi, Saif Hasan, Subodh Iyengar, Dan Johnson, Bingzhe Liu, Regina Ren, et al. Collective communication for 100k+ gpus. *arXiv preprint arXiv:2510.20171*, 2025.
- [58] James Turnbull. *Monitoring with Prometheus*. Turnbull Press, 2018.
- [59] Guanhua Wang, Shivaram Venkataraman, Amar Phanishayee, Nikhil Devanur, Jorgen Thelin, and Ion Stoica. Blink: Fast and generic collectives for distributed ml. *MLSys*, 2020.
- [60] Shibo Wang, Jinliang Wei, Amit Sabne, Andy Davis, Berkin Ilbeyi, Blake Hechtman, Dehao Chen, Karthik Srinivasa Murthy, Marcello Maggioni, Qiao Zhang, et al. Overlap communication with dependent computation via decomposition in large deep learning models. In *ASPLOS*, 2022.
- [61] Yongji Wu, Yechen Xu, Jingrong Chen, Zhaodong Wang, Ying Zhang, Matthew Lentz, and Danyang Zhuo. Mccs: A service-based approach to collective communication for multi-tenant cloud. In *SIGCOMM*, 2024.
- [62] Guanbin Xu, Zhihao Le, Yinhe Chen, Zhiqi Lin, Zewen Jin, Youshan Miao, and Cheng Li. Autoccl: Automated collective communication tuning for accelerating distributed and parallel dnn training. In *NSDI*, 2025.
- [63] Tailing Yuan, Yuliang Liu, Xucheng Ye, Shenglong Zhang, Jianchao Tan, Bin Chen, Chengru Song, and Di Zhang. Accelerating the training of large language models using efficient activation rematerialization and optimal hybrid parallelism. In *ATC*, 2024.
- [64] Liangyu Zhao, Siddharth Pal, Tapan Chugh, Weiyang Wang, Jason Fantl, Prithwish Basu, Joud Houry, and Arvind Krishnamurthy. Efficient direct-connect topologies for collective communications. In *NSDI*, 2025.
- [65] Hao Zheng, Chengyuan Huang, Xiangyu Han, Jiaqi Zheng, Xiaoliang Wang, Chen Tian, Wanchun Dou, and Guihai Chen. μ mon: Empowering microsecond-level network monitoring with wavelets. In *SIGCOMM*, 2024.
- [66] Yibo Zhu, Haggai Eran, Daniel Firestone, Chuanxiong Guo, Marina Lipshteyn, Yehonatan Liron, Jitendra Padhye, Shachar Raindel, Mohamad Haj Yahia, and Ming Zhang. Congestion control for large-scale rdma deployments. In *SIGCOMM*, 2015.

Appendix

A NCCL SM utilization

Table 1 presents the SM utilization of different P2P workloads in NCCL. Experimental data in Table 1 shows that intra-host p2p (32SMs by default) occupies up to 13.2% SM while inter-host p2p (2SMs by default) consumes 1.8% SM. Meanwhile, single-node alltoall (28SMs by default) uses up to 13.1% SM whereas two-nodes alltoall (4SMs by default) utilizes 2.3% of total SM.

Table 1: NCCL SM utilization of P2P workloads.

Workload Metric	Intra-host P2P	Inter-host P2P	8 ranks alltoall	16 ranks alltoall
Default SMs	32	2	28	4
SM utilization (%)	13.2	1.8	13.1	2.3

B Structure of SyncFifo

SyncFifo maintains the state required for in-place chunk retransmission. Table 2 details the structure. Specifically, fifoHead supports CTS synchronization, while the sender rank relies on restartPos and errorPort to determine the resumption point and identify the faulty link.

Table 2: Structure of SyncFifo.

Object	Meaning
fifoHead	Synchronize the offsets of CTS message.
restartPos	Represent the retransmission chunk (<i>done</i>).
errorPort	Indicate the abnormal port.

C Analysis of data size in PP

In pipeline parallelism (PP), activations are communicated between adjacent pipeline stages. The size of these communicated tensors is determined by the hidden dimension, sequence length, and micro-batch size, which together often lead to communication volumes exceeding tens of megabytes. Formally, consider a transformer-based model with hidden size H , sequence length L , and micro-batch size B . The activation tensor exchanged between two consecutive pipeline stages can be expressed as

$$\mathcal{A} \in \mathbb{R}^{B \times L \times H}. \quad (1)$$

Assuming a data type with precision p bytes (e.g., $p = 2$ for FP16), the communication size per pipeline transfer is

$$S_{pp} = B \times L \times H \times p. \quad (2)$$

For modern large-scale language models, H typically ranges from several thousand to over ten thousand, while L is often set to hundreds or thousands. Even with a modest micro-batch

size, the resulting communication volume quickly exceeds 32 MB. For example, with $B = 4$, $L = 2048$, $H = 8192$, and FP16 precision, we obtain

$$S_{pp} = 4 \times 2048 \times 8192 \times 2 \approx 128 \text{ MB}. \quad (3)$$

In addition, pipeline parallelism typically employs multiple micro-batches to improve device utilization. Let M denote the number of micro-batches in a pipeline schedule. The cumulative communication volume per iteration then scales linearly with M :

$$S_{pp}^{\text{iter}} = M \times S_{pp}. \quad (4)$$

These factors jointly explain why inter-stage communication in PP systems frequently involves data transfers significantly larger than 32 MB, making communication efficiency a critical bottleneck in large-scale pipeline-parallel training.

D Experiment Settings

The training hyperparameters and VCCL settings of our evaluation (§4) are listed in Table 3.

Table 3: Training hyperparameters and VCCL settings.

Category	Setting	Value
Training	Global batch size	512
	Micro batch size	1
	DP size	8
	TP size	2
	PP size	4
	Attention head number	64
	Sequence length	2048
	layers_per_virtual_pipeline_stage	2
	Learning rate	1.5e-4
	Optimizer	Adam
	Precision	BF16
VCCL	IB_TIMEOUT	18
	IB_RETRY_CNT	7
	Window size	8
CC traffic generator	Message size	4G
	QP number	2
	Channel number	32

E Performance gain of SM-free

The 5% gain stems from reduced SM contention and the elimination of scheduling-induced stragglers. For example, in 1F1B, the GEMM kernel `nvjet_tst_256x128_64x4_1x2_h_bz_coopA_TNT` launches 132 thread blocks, each with 12 warps, on a Hopper GPU with 132 SMs. In isolation, this kernel naturally forms one wave of execution, with roughly one GEMM block assigned to each SM. However, when it co-runs with the NCCL SendRecv kernel, which launches

two blocks with 20 warps each, the NCCL blocks can be co-resident with GEMM blocks on two of the SMs. On these SMs, the 12 GEMM warps no longer have exclusive access to the SM’s warp schedulers and execution resources; instead, they share and compete for scheduler issue slots, memory pipelines, and other SM resources with the 20 communication warps. Since the GEMM kernel cannot complete until all of its blocks finish, these few delayed GEMM blocks become tail stragglers and extend the kernel’s critical path. VCCL avoids this interference by preventing the communication kernel from being scheduled onto GEMM-occupied SMs, thereby removing the straggler-induced stalls that would otherwise delay GEMM completion.

F Resource Consumption of VCCL

Table 4 shows the kernel invocation and SM utilization of inter-host P2P communication using NCCL and VCCL. The second column in the table represents the invoked kernels and their respective percentage of executing duration, which are collected by Nsight. Obviously, NCCL invokes a `send/recv` kernel `ncclDevKernel_SendRecv` to perform the reduction-free P2P communication, which accounts for 68.3% execution time and 3.2% SM usage. Instead, except for two inevitable NCCL-Tests kernels (i.e., `prepareInput2` for data preparation and `verifyPrepare` for result verification), VCCL does not launch any GPU kernels to perform the P2P communication, thereby consuming lower SM utilization than NCCL. Note that in practical LLM training, `prepareInput2` and `verifyPrepare` will not be invoked, such that VCCL’s SM-free P2P can achieve almost zero SM consumption, which can be scheduled in computation to accelerate the LLM training process.

Table 4: Kernel invocation and SM consumption.

	Operations (percentage)	SM utilization
NCCL	<code>ncclDevKernel_SendRecv</code> (68.8%)	3.2%
	<code>verifyPrepare</code> (27.0%)	
	<code>prepareInput2</code> (3.5%)	
	[CUDA memset] (0.4%)	
	[CUDA memcpy Host-to-Device] (0.4%)	
VCCL	<code>verifyPrepare</code> (86.1%)	2.8%
	<code>prepareInput2</code> (11.4%)	
	[CUDA memset] (1.3%)	
	[CUDA memcpy Host-to-Device] (1.2%)	

Figure 17 compares the CPU utilization of a GPU server using NCCL and VCCL, respectively. Although VCCL’s SM-free design offloads P2P from GPU kernels to CPU proxies, it only introduces about 2% more CPU utilization than NCCL. The result validates VCCL’s deployability and feasibility in real-world GPU cluster, where a low CPU utilization is essential to maintain the CC efficiency (discussed in §6).

Table 5 reports the system overhead introduced by the online monitor. Enabling the monitor increases CPU utilization from 9.32% to 21.1%, corresponding to an additional 11.8%

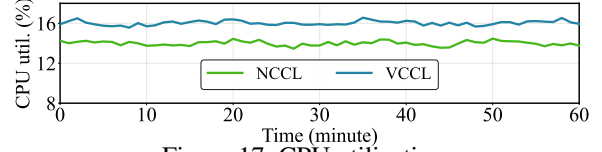


Figure 17: CPU utilization.

CPU overhead, while memory utilization rises marginally from 1.7% to 2.1%. Overall, the online monitor incurs moderate CPU overhead and negligible memory overhead, indicating that its resource consumption remains acceptable for practical deployment.

Table 5: System overhead of online monitor.

	Scheme	w/o monitor	w/ monitor
Hardware			
CPU utilization		9.32%	21.1%
Memory utilization		1.7%	2.1%

G Resilience under multi-port failures.

Meanwhile, to evaluate the robustness of our primary-backup mechanism under multi-port failures, we conduct an `AllReduce` stress test between two nodes by progressively disabling RNIC ports. Figure 18 illustrates the bandwidth dynamics across different failure scenarios. In Phase 0, the system maintains a stable baseline bandwidth of 450 Gbps. Upon a single port failure (RNIC 0) in phase 1, traffic successfully migrates to the backup QP on RNIC 1, with bandwidth stabilizing at 350 Gbps. This drop primarily results from physical port sharing and increased PCIe contention as RNIC 1 services two GPUs. Subsequently, disabling a second port (RNIC 2) in phase 2 triggers a performance degradation to 190 Gbps, characterized by severe instability and jitter. After careful investigation, we conclude this to congestion collapse, where the asymmetric topology induces many-to-one incast, triggering frequent Priority Flow Control (PFC) [25] frames and RDMA backpressure. Notably, the disabling of the third port (RNIC 4) causes no further performance drop. This indicates that the bottleneck has shifted from available link capacity to network-wide congestion. At this stage, the remaining active paths are fully saturated by the overhead of managing congestion, preventing any further throughput degradation despite fewer available links. Finally, bringing up all failed ports in phase 4 allows the bandwidth to recover to 450 Gbps, demonstrating that VCCL can effectively tolerate and recover from multi-port link failures.

H Wisdom of Sliding window size

To evaluate the observability, we generate a background P2P workload between a pair of servers, and insert a disturbance traffic to compete for bandwidth, making the background workload converge to a lower and stable throughput. Figure

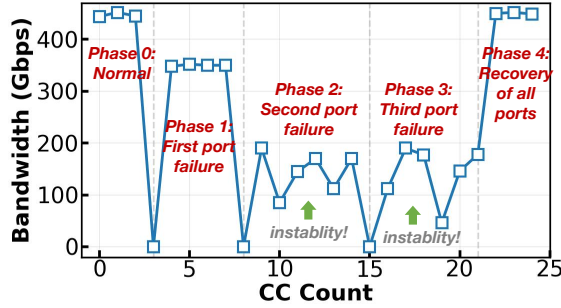


Figure 18: Tolerance to multi-port failures.

19 presents the runtime throughput in a $10\mu s$ granularity with different window sizes. Especially, when window size is 1, it equals to the naive per-message scheme. We notice VCCL can capture the transient $O(\mu s)$ throughput changes, but shows a very fluctuating measuring result with per-message scheme. With a large window size of 32, VCCL can smooth out the fluctuation and reflects more accurate results closer to the ground truth, but fails to perceive the instantaneous changes. For example, when disturbance traffic arrives at $100\mu s$, per-message scheme shows an intense fluctuation, but a large window size directly drops to the new converged throughput. Therefore, we set the window size as 8 to achieve the tradeoff between measuring accuracy and fluctuation sensitivity in our environment.

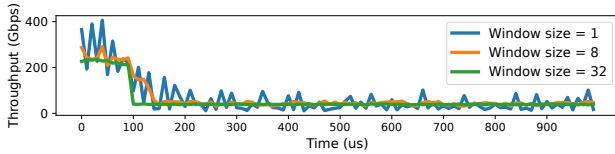


Figure 19: Monitor results with different window sizes.

I Full-stack Troubleshooting Platform

Figure 20 presents the system architecture, organized into three hierarchical layers: 1) Object Layer: covers both physical and logical resources, from hardware (e.g., GPUs, network, storage) to cloud platforms and applications, enabling end-to-end observability. 2) Monitoring and Acquisition Layer (Middle): bridges resources and decisions via specialized collectors. Standard tools (e.g., dcgm [46], Prometheus [58]) handle general metrics, while VCCL provides NIC-level troubleshooting and $O(\mu s)$ monitoring capabilities, capturing transient network jitters that are invisible to coarse-grained approaches (e.g., SNMP [26]). VCCL also supports customizable application-level metrics (e.g., opFuncTimes, ccType). 3) Decision and Interaction Layer (Top): an intelligent agent aggregates multi-dimensional data via Elastic stacks [20] and platform APIs, driving visualization, alerting, ticketing, and automated fault isolation.

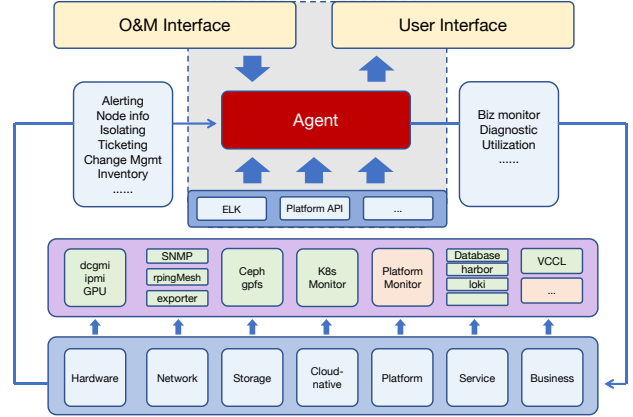


Figure 20: Full-stack trouble shooting platform.

J Effectiveness of memory optimization

To evaluate the effectiveness of the dynamic memory pool in VCCL, we systematically monitor the memory footprint of both NCCL and VCCL under a wide range of configurations. In particular, we consider two representative dense models, GPT-2 32B and 70B, as well as two MoE models, Qwen3-30B-A3B and 235B-A22B. As shown in Figure 21, VCCL achieves up to 26.7% memory reduction compared to NCCL. This result highlights VCCL's strong capability in significantly reducing HBM consumption, thereby enabling users to allocate more resources to model execution and focus on higher-level application logic rather than communication-induced memory overhead.

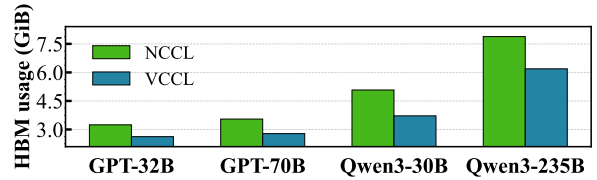


Figure 21: Memory optimization.

K Additional Experience

Proactive isolation is necessary. To mitigate the impact of impending hardware failures, we adopt a proactive isolation policy driven by early warning signals. For RNIC, we observe that transient link instabilities often precede hard failures. Specifically, if an RNIC exhibits two minor flap events within a five-minute window, we immediately mark the host as unhealthy and proactively isolate it. The isolation is enforced after the next checkpoint completes, at which point the faulty network interface is replaced. This approach allows us to avoid disruptive failures while preserving application progress. For GPUs, we distinguish between failure modes with different risk profiles. Hardware integrity violations, such as ECC errors, trigger immediate isolation due to their strong

correlation with irreversible failures and data corruption. In contrast, performance degradation caused by environmental factors—such as fan anomalies or transient thermal issues—is handled more conservatively. As long as the number of such degradation events does not exceed five occurrences within a 24-hour period, we do not isolate the GPU, allowing the system to tolerate benign fluctuations without unnecessary disruption.

Congestion control. As most CC primitives do not incur severe network congestion, (e.g., ring-based `allreduce`, `allgather`, `reducescatter`), we draw some lessons about how to cooperate VCCL with congestion control, for example, DCQCN [66]. On the one hand, when our customers are training dense models that incur mild incasts, tuning DCQCN parameters does not show an obvious improvement on training performance. As our network architecture already provides sufficient bidirectional bandwidth without oversubscription, we can disable the congestion control and only enable PFC [25] to ensure a lossless RoCEv2 environment, like the other . On the other hand, when our customers are training MoE models with significant `alltoall` workloads that generate severe incasts, it becomes necessary to tune performant DCQCN parameters for better CC performance. As a feasible solution, we can drive the DCQCN parameters in throughput-friendly or delay-friendly directions by flow distribution at runtime [6].